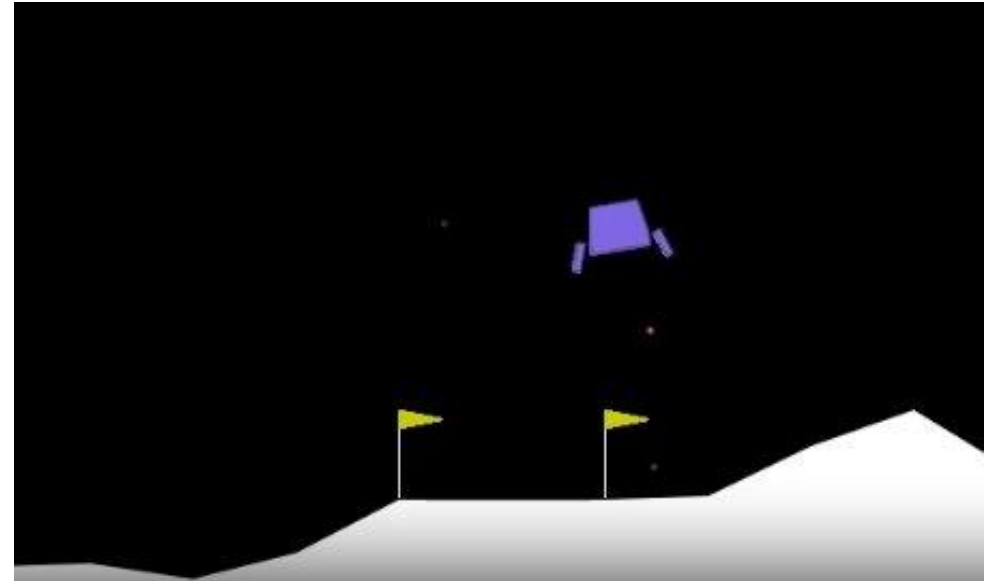


DQN 계열 알고리즘을 활용한 LunarLander- 성능 개선

김민재

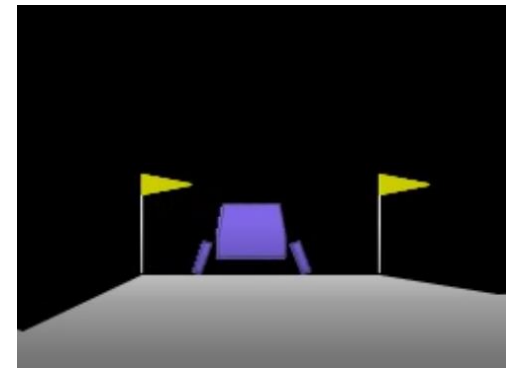
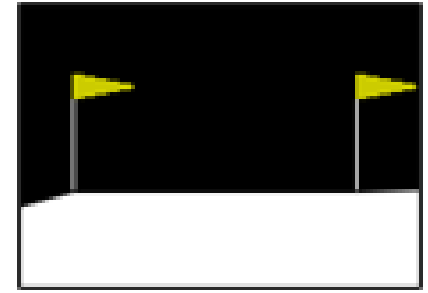
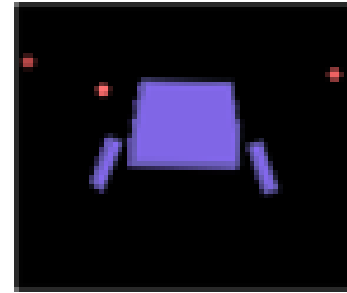
LunarLander

- 강화학습에서 가장 널리 쓰이는 제어(Control) 문제
- 간단한 2D 물리환경 + 명확한 목표(착륙)
- RL 알고리즘 비교/시각화하기에 최적의 benchmark
- 실제 로켓 착륙 시스템과 유사한 구조



LunarLander

- 목표: 착륙선을 중앙 패드에 안정적으로 착륙시키는 것
- 기반 엔진: Box2D
- 제어해야 할 물리 요소: 중력, 관성, 회전, 충돌 등
- 행동(action) 4개
 - 0: do nothing
 - 1: fire left engine
 - 2: fire main engine
 - 3: fire right engine



LunarLander

- 상태 8개
 - x 위치
 - y 위치
 - x 속도
 - y 속도
 - 각도
 - 각속도
 - 왼쪽 다리 접촉
 - 오른쪽 다리 접촉
- 보상 구조
 - 패드에 가까워지면 +
 - 속도 낮추면 +
 - 자세(각도) 안정적일수록 +
 - 착지 다리 접촉 10점
 - 성공 착륙 +100 이상
 - 충돌 시 -100 (종료)
- 에피소드가 200점 이상을 받으면 솔루션으로 간주

프로젝트 개요

- 본 프로젝트는 LunarLander-v2 환경에서 DQN 알고리즘의 개선하고 성능을 비교하는 것을 목표로 한다.
- 기본 DQN을 시작으로 Double DQN, Dueling DQN, D3QN(Dueling + Double)을 구현하고 1000 에피소드 동안 동일 조건에서 학습하였다.

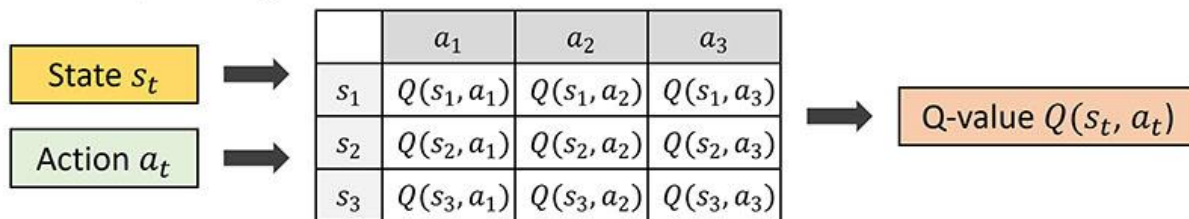
알고리즘	목적	특징
DQN	기본 구조	Q-learning + Neural Network
Double DQN	과대평가 방지	Action selection / evaluation 분리
Dueling DQN	가치 기반 분해	안정적 가치 추정
D3QN	Double + Dueling 결합	가장 안정된 성능

문제 정의 / 해결 목표

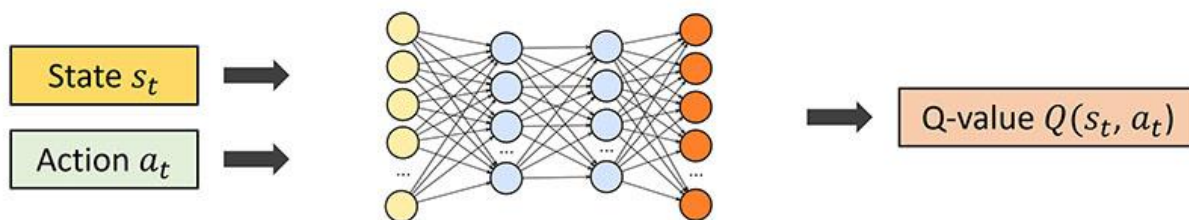
- LunarLander 환경은 초기 상태에 따라 난이도가 달라지는 비선형 제어 문제이다.
- 단순 DQN 알고리즘은 Q-value 과대평가와 학습 불안정성 문제가 존재한다.
- 본 프로젝트는 강화학습 알고리즘의 **성능 향상 및 안정화**를 목표로 한다.
- **해결 목표:**
 - DQN 계열 알고리즘을 구현 및 재현
 - 알고리즘별 학습 성능 비교
 - 가장 높은 안정성과 평균 리워드를 갖는 알고리즘 도출

DQN

Classic Q-learning



Deep Q-learning



- 핵심 아이디어

- Q-learning을 신경망으로 근사하여 고차원 상태에서도 Q-table 없이 학습 가능하게 만든 알고리즘
 - Experience Replay(transition replay buffer에 저장 학습)
 - Target Network
- 이 두 기법으로 학습 불안정성을 크게 줄였음

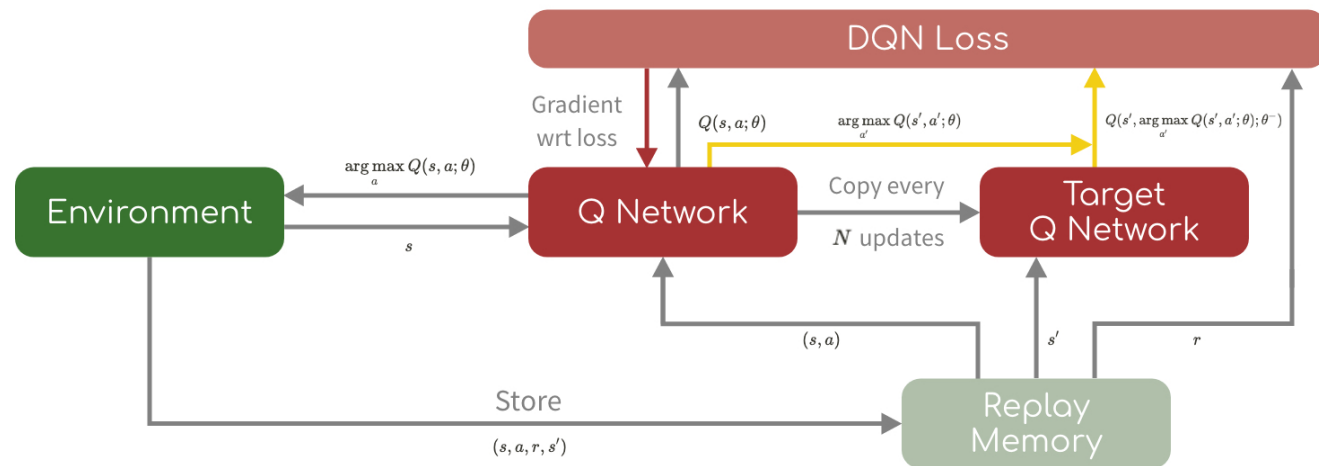
- 장점

- 구현이 간단하고 빠르게 학습됨
- 초기 성능이 잘 나오는 편

- 단점

- Q-value 과대평가(overestimation) 문제 존재
→ 실제보다 더 높은 Q를 예측해 정책이 불안정해짐

Double DQN (DDQN)



왜 등장했나?

- DQN은 다음 값을 계산할 때 Q-value가 불필요하게 크게 평가(overestimation) 되는 문제가 발생한다.
- Double DQN 핵심 해결책
 - Q값을 계산할 때 행동 선택과 행동 평가를 분리한다.
 - 행동 선택 (argmax) $\rightarrow$ policy_net(q_net)
 - 행동 가치 평가 (Q-value) $\rightarrow$ target_net

장점

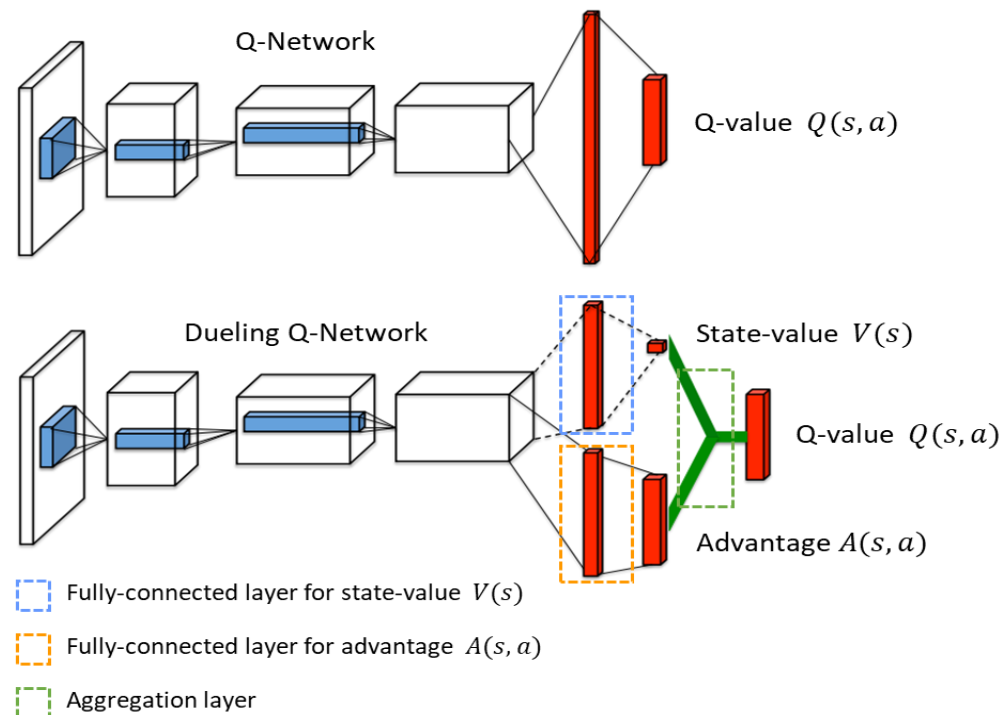
- Q-value 안정성 증가
- 학습 후반 성능 향상

단점

- 초기 학습 속도는 다소 느릴 수 있음

Dueling DQN

- 접근 방법
 - state에 따라 “행동 선택의 중요도”가 다르다.
 - DQN은 모든 (state, action)에 대해 Q를 모두 계산해야 함
→ 비효율적이고 상태 가치 추정이 어려움
- Dueling 구조의 핵심 아이디어
 - 상태 가치 $V(s)$
 - 행동 이점(advantage) $A(s,a)$로 분해하여 학습한다.

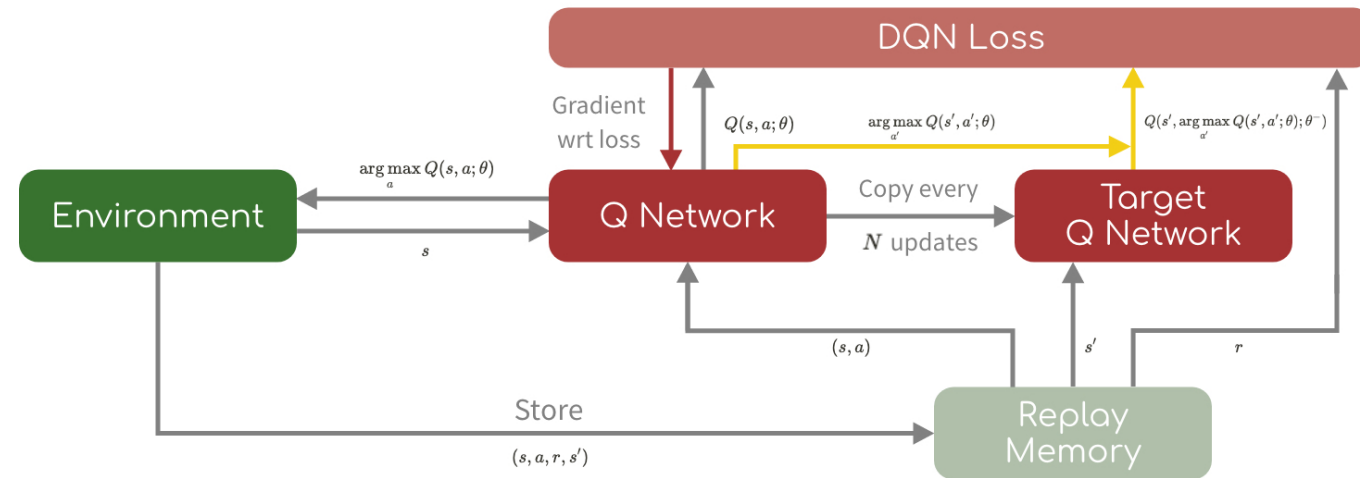


Dueling DQN

- 장점
 - 상태 가치 $V(s)$ 를 잘 학습하여 정책 안정성 향상
 - 중요하지 않은 행동들에 덜 민감 $\rightarrow$ 학습 효율 증가
 - 리워드 변동성이 큰 환경에서 더 유리
- 단점
 - 구조가 복잡해 초기 학습이 느릴 수 있음

D3QN (Double + Dueling) — 최종 알고리즘

- 등장 배경
 - Double은 과대평가 문제 해결
 - Dueling은 가치 분해로 안정성 개선
→ 둘 다 보완 관계
- 장점
 - DQN보다 훨씬 안정적
 - Double보다 수렴 속도가 빠르고 더 좋은 정책 획득
 - Dueling보다 안정성 우수
 - 가장 높은 평균 리워드



개발 환경

- Google Colab 기반 개발
- Python 3.12
- PyTorch 2.x
- Gymnasium 0.29.1
- Box2d-py 2.3.5
- Numpy 1.26.4
- Pygame 2.5.2
- CPU/GPU: Colab T4 GPU 사용

학습 설정(Hyperparameters)

- 학습 에피소드: 500
- 최대 스텝: 1000
- Batch size: 64
- Replay buffer size: 50,000
- γ (할인율): 0.99
- ϵ -greedy: 1.0 $\rightarrow$ 0.05 (300 episode 동안 선형 감소)
- Optimizer: Adam(lr = 1e-3)
- Target update interval: 10 episodes

기본 dqn 구조

```
class QNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim)
        )

    def forward(self, x):
        return self.net(x)
```

- Q-Network 구조
 - 2-layer MLP (128->128)
 - 입력: state
 - 출력: 각 행동 Q값
 - 활성화함수: ReLu
- Replay Buffer
 - transition(s, a, r, s', done) 저장
 - 배치 샘플링으로 데이터 상관성 제거
- ϵ -greedy 탐색
 - 초기 $\epsilon=1.0 \rightarrow 300$ 에피소드 동안 0.05까지 선형 감소
- Target Network
 - policy_net을 일정 타임마다 복사
 - 학습 안정화 목적

학습 파이프라인

- Replay Buffer 기반 학습
- Batch 단위로 Q-target 계산
- Loss 계산
- 네트워크 업데이트
- Target network 동기화 (10ep)

Double DQN 구현

변경점

- 행동 선택 : ddqn_policy_net
- 행동 평가 : ddqn_target_net

```
with torch.no_grad():  
    # 1) 다음 상태에서 최적 행동은 policy_net으로 선택  
    next_q_policy = ddqn_policy_net(next_states)  
    next_actions = next_q_policy.argmax(dim=1, keepdim=True) # [B,1]  
  
    # 2) 그 행동의 Q값은 target_net으로 평가  
    next_q_target = ddqn_target_net(next_states).gather(1, next_actions)  
  
    target = rewards + gamma * (1 - dones) * next_q_target
```


Dueling dqn 구현

- 상태 입력(8) $\rightarrow$ 128 $\rightarrow$ 128 (ReLU)
- 128 특징 벡터 변환
- Value Stream(128 $\rightarrow$ 64 $\rightarrow$ 1),
Advantage Stream(128 $\rightarrow$ 64 $\rightarrow$ action_dim) 분리
- 출력 조합 $\rightarrow$ Q(s,a) 생성

```
class DuelingQNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()

        # 공통 Feature Extractor
        self.feature = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU()
        )

        # V(s): State Value Stream
        self.value_stream = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

        # A(s, a): Advantage Stream
        self.advantage_stream = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, action_dim)
        )

    def forward(self, x):
        x = self.feature(x)

        value = self.value_stream(x)  # [B, 1]
        advantage = self.advantage_stream(x)  # [B, action_dim]

        # Q(s,a) = V(s) + A(s,a) - mean(A(s,a))
        q_values = value + advantage - advantage.mean(dim=1, keepdim=True)
        return q_values
```

D3QN 구현

- 네트워크 구조: Dueling DQN과 동일
- Target 계산: Double DQN 방식 사용

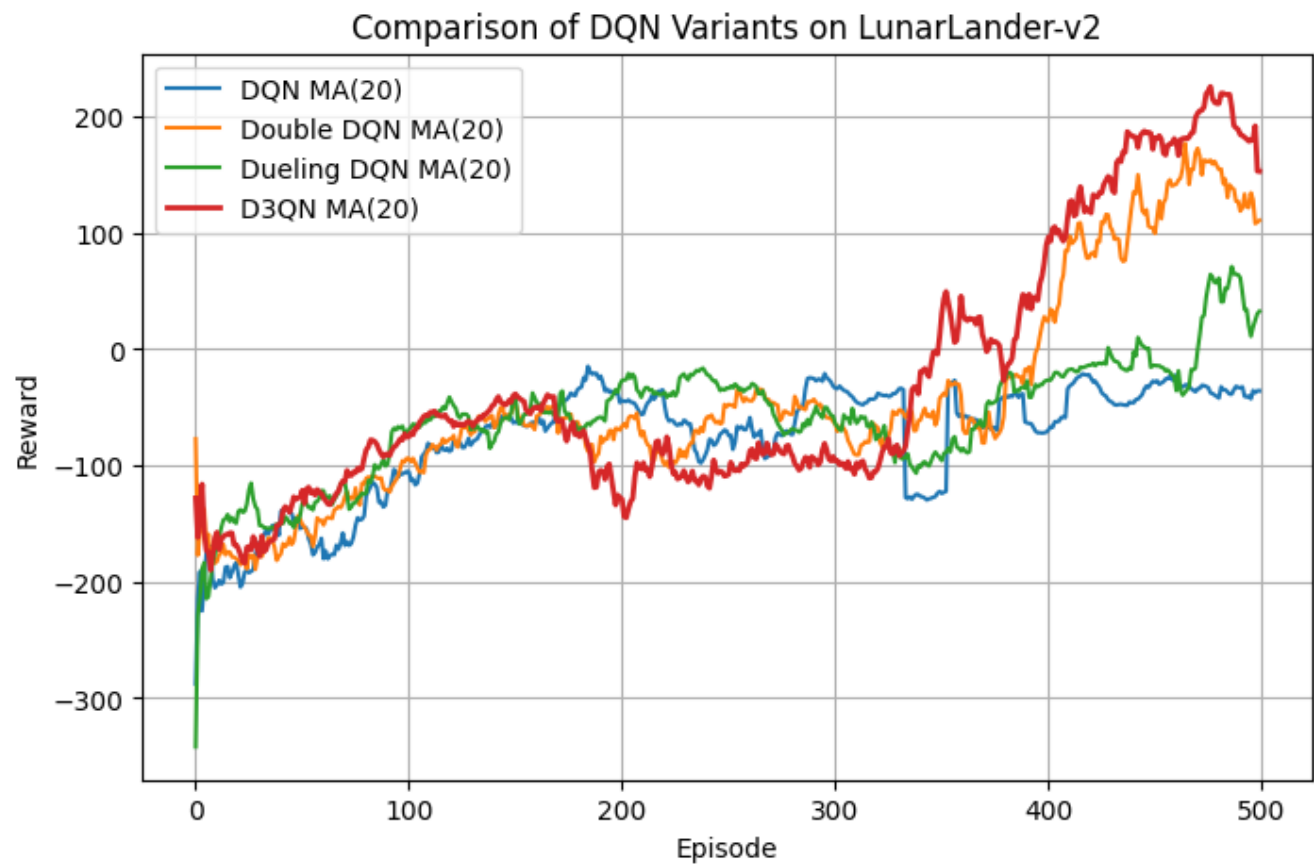
모델 저장 & 평가 과정

- 에피소드별 총 리워드 `total_reward` 기록
- 현재 에피소드 리워드가 **역대 최고 리워드보다 크면**
 - 그 시점의 `policy_net` 가중치를 저장

```
if total_reward > best_ddqn_reward:  
    best_ddqn_reward = total_reward  
    best_ddqn_state = ddqn_policy_net.state_dict()
```

- 학습이 끝난 후, `best_state`를 다시 불러와서 Best Model 기준으로 평가

성능 평가



성능 평가

```
DQN   last 100-ep mean: -34.14601145178673
DDQN  last 100-ep mean: 122.12938044631719
Duel  last 100-ep mean: 7.108534539592752
D3QN  last 100-ep mean: 169.61269425289487
```

- 학습 마지막 100 에피소드

- DQN 평균 ≈ -34

- 랜딩을 거의 못 하고 있음 (거의 항상 충돌/실패)

- DDQN 평균 ≈ 122

- 성공/실패가 섞여 있지만, 전반적으로 착륙을 자주 성공하는 수준

- Dueling 평균 ≈ 7

- 현재 파라미터/에피소드 수로는 Dueling 구조가 충분히 학습되지 않았음을 의미

- D3QN 평균 ≈ 170

- 수렴하고 안정적인 정책 측면에서는 D3QN이 가장 우수하다는 것을 보여줌

성능 평가

- Best_model로 100 에피소드 평균 값
- DQN 평균 ≈ -28
 - 랜딩을 거의 못 하고 있음 (거의 항상 충돌/실패)
- DDQN 평균 ≈ 90
 - 성공/실패가 섞여 있지만, 전반적으로 착륙을 자주 성공하는 수준
- Dueling 평균 ≈ -33
 - DQN이랑 비슷하거나 더 나쁜 수준 $\rightarrow$ 사실상 제대로 못 배움
- D3QN 평균 ≈ 51
 - 어느 정도 성능은 있는데, 아직 DDQN만큼 안정적이지 않음

=== Evaluation Results ===

DQN Mean Reward : -28.43

DDQN Mean Reward : 90.05

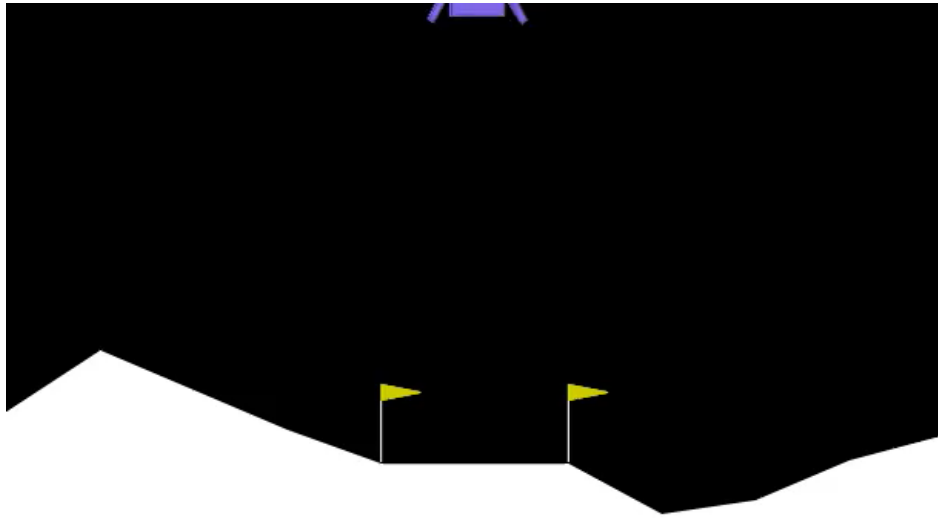
Dueling Mean Reward : -33.01

D3QN Mean Reward : 50.85

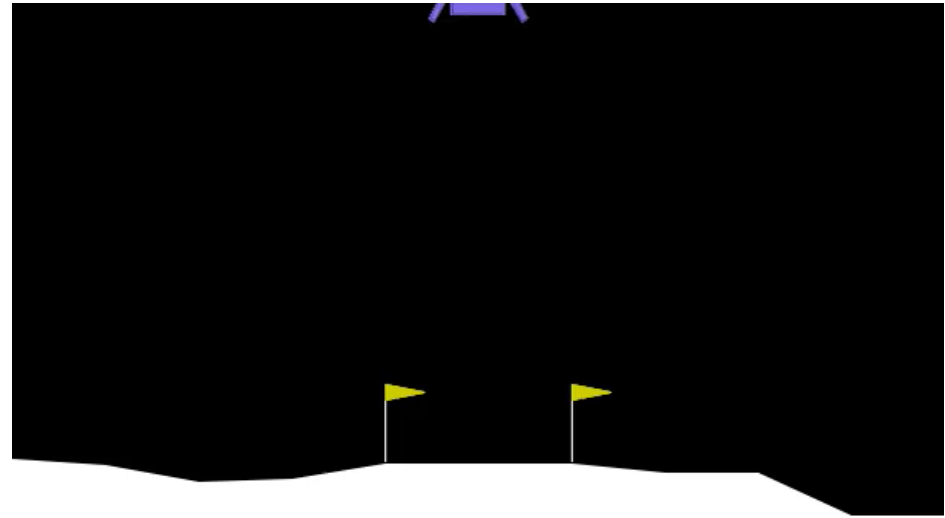
성능 평가 개선 할 점

- 에피소드 수(1000)가 아직 부족
 - Dueling / D3QN은 구조가 복잡해서 DQN/DDQN보다 초반 수렴이 느린 편이라 1000 episodes에서는 DDQN이 더 좋아 보일 수 있음.
- `best_model` 저장 위치가 진짜 “후반부 잘 학습된 모델”이 아닐 가능성
 - 학습 초반에 우연히 한 번 높은 리워드를 찍고 그때의 모델이 `best_state`로 저장됐을 수도 있음.
 - 특히 변동성이 큰 환경에서 자주 나오는 패턴.

일부 플레이 영상

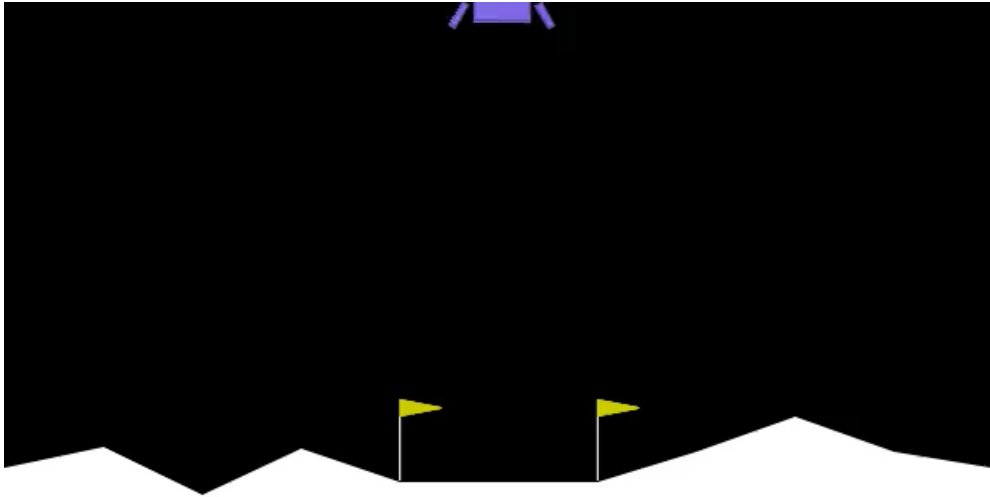


- Dqn

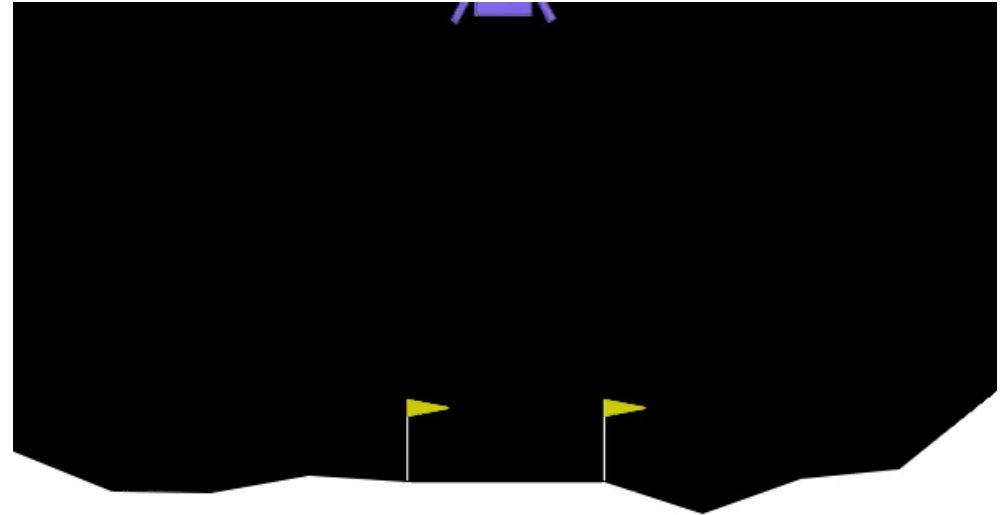


- Ddqn

일부 플레이 영상



- Duelling dqn



- D3qn

향후 목표

- Best_model 저장 방법 개선
 - 에피소드 단일 리워드 → “이동평균 기준”으로 바꾸기
 - 한 에피소드 스파이크 대신
최근 N 에피소드 평균 리워드(MA)를 기준으로 best 모델을 저장.
- Rainbow DQN 적용
 - DeepMind가 발표한 DQN의 “올인원” 버전.
 - 포함된 7요소:
 - Double DQN
 - Prioritized Replay (PER)
 - Dueling Network
 - Multi-step Learning (n-step)
 - Distributional RL (C51)
 - NoisyNet exploration
 - Rank-based updates